

Scale Down Patroni

Overview

If you are reading this runbook you are most likely tasked to scale down our Patroni cluster. There can be several reasons of us wanting to scale our cluster down such as cost saving or rebuilding a node. Use this runbook as a guidance on how to safely scale down a Patroni cluster – which will eventually result in removing a patroni node.

Pre-requisite

- Patroni This runbook assumes that you know what Patroni is, what and how we use it for and possible consequences that might come up if we do not approach this operation carefully. This is not to scare you away, but in the worst case: Patroni going down means we will lose our ability to preserve HA (High Availability) on Postgres. Postgres not being HA means if there is an issue with the primary node Postgres wouldn't be able to do a failover and GitLab would shut down to the world. Thus, this runbook assumes you know this ahead of time before you execute this runbook.
- Chef You are also expected to know what Chef is, how we use it in production, what it manages and how we stop/start chef-client across our hosts.
- Terraform You are expected to know what Terraform is, how we use it and how we make change safely (`tf plan` first).

Scope

This runbook is intended only for one or more **read** replica node(s) of Patroni cluster.

Mental Model

Let's build a mental model of what all are at play before you scale down Patroni cluster.

- We have a Patroni cluster up and running in production
- The replica nodes are taking read requests and processing them
- The fact that we have a cluster, it means the cluster might decide to promote any replica (except those with the tag `nofailover: true`) to primary (can be the target replica node you are trying to remove)
- There is chef-client running regularly to enforce consistency
- The cluster size is Terraform'd via this

What this means is that we need to be aware of and think of:

1. Choose a node that is not already the primary or specially configured.
 1. You can determine the current primary by running `gitlab-patronictl list` on any Patroni node in the cluster
 2. It should have `{}` for options (e.g., `1 = {}`), meaning that it's just a regular replica.
 3. It's OK to take nodes from the middle as our terraform module is designed to handle gaps in the numbering.
2. Stop chef-client so that any change we make to the replica node and Patroni doesn't get overwritten
3. Prevent the target replica node from getting promoted to primary
4. Take the node out of loadbalancing to drain all connections and then take the replica node out of the cluster
 1. There is a known issue with the Rails processes not refreshing their load balancer DNS cache and this may account for delays in draining connections. You may need to wait until the next deployment for all Rails processes to be restarted to see all connections drained from the replica. The rails processes should technically be resilient to replicas going down but waiting until connections drain would be the safest option.
5. Make a Terraform change and merge.

Execution

Preparation

1. You should do this activity in a CR (thus, allowing you to practice all of it in staging first). Here are examples of past change requests to scale down replicas:
 1. Patroni Main:
 1. GSTG
 2. GPRD
 2. Patroni CI:
 1. GSTG
 2. GPRD
2. Make sure the replica you are trying to remove is NOT the primary, by running `gitlab-patronictl list` on a patroni node
3. Pull up the Host Stats Grafana dashboard and switch to the target replica host to be removed. This will help you monitor the host.

Step 1 - Stop chef-client

- On the replica node run: `sudo chef-client-disable "Removing patroni node: Ref issue prod#xyz"`

Step 2 - Take the replicate node out of load balancing

If clients are connecting to replicas by means of service discovery (as opposed to hard-coded list of hosts), which is the case for all Rails processes connecting to Patroni Main and Patroni CI, you can remove a replica from the list of hosts used by the clients by tagging it as not suitable for failing over (`nofailover: true`) and load balancing (`noloadbalance: true`). (If clients are configured with `replica.patroni.service.consul`. DNS record look at this legacy method)

1. Add a tags section to `/var/opt/gitlab/patroni/patroni.yml` on the node:

```
tags:
  nofailover: true
  noloadbalance: true
```

2. `sudo systemctl reload patroni`
3. Test the efficacy of that reload by checking for the node name in the list of replicas:

```
dig @127.0.0.1 -p 8600 db-replica.service.consul. SRV
```

If the name is absent, then the reload worked.

4. Wait until all client connections are drained from the replica (it depends on the interval value set for the clients), use this command to track number of client connections:

```
for c in /usr/local/bin/pgb-console*; do $c -c 'SHOW CLIENTS;' | grep gitlabhq_production
```

It can take a few minutes until all connections are gone. If there are still a few connections on pgbouncers after 5m you can check if there are actually any active connections in the DB (should be 0 most of the time):

```
gitlab-psql -qtc "SELECT count(*) FROM pg_stat_activity
WHERE pid <> pg_backend_pid()
AND datname = 'gitlabhq_production'
AND state <> 'idle'
AND username <> 'gitlab-monitor'
AND username <> 'postgres_exporter';"
```

NOTE: There is a known issue with the Rails processes not refreshing their load balancer DNS cache and this may account for delays in draining connections. If this still isn't fixed, you may need to wait until the next deployment for all Rails processes to be restarted to see all connections drained from the replica. The rails processes should technically be resilient to replicas going down but waiting until connections drain would be the safest option.

You can see an example of taking a node out of service in this issue.

Step 3 - Stop patroni service on the node

Now it is safe to stop the Patroni service on this node:

```
sudo systemctl stop patroni
```

This will also stop Postgres and thus terminate all remaining db connections if there are still some. With the Patroni service stopped, you should see this node vanish from the cluster after a while when you run `gitlab-patronictl list` on any of the other nodes. We have alerts that fire when Patroni is deemed to be down. Since this is an intentional change - either silence the alarm in advance and/or give a heads up to the EOC.

Step 4 - Terraform change to decrease the count

1. Choose a node that is not already the primary or specially configured.
 1. You can determine the current primary by running `gitlab-patronictl list` on any Patroni node in the cluster
 2. It should have `{}` for options, meaning that it's just a regular replica.
 3. It's OK to take nodes from the middle as our terraform module is designed to handle gaps in the numbering.
2. Remove the `nodes` entry for the relevant node in the relevant cluster in terraform.
3. `tf plan` to make sure the plan output looks what you expect
4. File an MR, get it reviewed/merged then monitor the `ops.gitlab.net` pipeline to review the plan and apply it (apply job will be in blocked state).
5. Wait until the node gets removed and torn down. (Validate it in GCP)

Take a look at a sample code change.

Automation Thoughts

An excellent automation initiative is underway: https://ops.gitlab.net/gitlab-com/gl-infra/db-ops/-/blob/master/ansible/playbooks/roles/patroni/tasks/top_traffic.yml.

Reference

Majority of this runbook was written based on the content we have in: `patroni-management`. But since we are going through a sprint of creating separate runbook for each activity, it makes sense to separate out the individual type of work into its own runbook.